

dnipro

Symbolic Equation-Based MNA Framework
for SPICE-like Circuit Modeling in Python and SymPy

Reference Manual

Version 0.1.0

Package name:	dnipro
Implementation language:	Python 3.11+
Core symbolic engine:	SymPy $\geq$ 1.14
External verifier:	ngspice (optional)
Entry point:	dnipro analyze

May 10, 2026

Contents

I	User Guide	4
1	Introduction	4
1.1	What dnipro is	4
1.2	What dnipro is not	4
1.3	Relationship to the specification	4
1.4	Relationship to the User's Guide	4
2	Installation and Setup	5
2.1	Requirements	5
2.2	Installing from source	5
2.3	Verifying the installation	5
2.4	Running the test suite	5
3	Quick Start	5
3.1	Example 1: Resistive voltage divider (time domain)	5
3.2	Example 2: RC low-pass filter (Laplace domain)	6
3.3	Example 3: Numeric substitution	6
3.4	Example 4: Classical linear MNA report	6
3.5	Example 5: Descriptor MNA report	7
4	Command-Line Interface	7
4.1	General form	7
4.2	Options	7
4.3	Exit codes	8
4.4	Extended examples	8
5	Packaged Examples and Tutorials	8
5.1	pub1: small-signal VCCS example	9
5.2	MOSFET small-signal tutorial	9
6	Netlist Syntax	10
6.1	General rules	10
6.2	Node names	10
6.3	Component names	10
6.4	Supported element lines	10
6.5	Values	11
6.6	Diode model syntax	11
7	Source Functions	12
7.1	DC (constant)	12
7.2	SIN (sinusoidal)	12
7.3	PULSE	12
7.4	PWL (piecewise linear)	12
7.5	SQUARE (square wave)	12
7.6	SAW (sawtooth)	13

II Mathematical Reference	13
8 MNA Theory	13
8.1 Overview of Modified Nodal Analysis	13
8.2 Unknown vector	13
8.3 Residual vector	14
8.4 KCL residuals	14
8.5 Voltage-defined branches	14
8.6 Reference direction conventions	14
9 Component Equations	15
9.1 Resistor	15
9.2 Capacitor	15
9.3 Inductor	15
9.4 Voltage source	15
9.5 Current source	15
9.6 VCVS (E source)	16
9.7 VCCS (G source)	16
9.8 CCCS (F source)	16
9.9 CCVS (H source)	16
9.10 Diode	16
10 Time Domain and Laplace Domain	16
10.1 Time-domain mode	16
10.2 Laplace-domain mode	17
10.3 Frequency substitution	17
11 Nonlinear Component: Diode	17
11.1 Design restriction	17
11.2 What is not implemented	17
11.3 Model parameters	17
11.4 Example	18
III Library API Reference	18
12 Package Overview	18
12.1 Public API	18
12.2 Package structure	18
13 Parser Module	19
13.1 parse_netlist_text	19
13.2 parse_netlist_file	19
13.3 parse_value	19
13.4 parse_source	20
14 Data Model	20
14.1 Circuit	20
14.2 Component classes	21
14.3 DiodeModel	21
14.4 ResidualSystem	21
14.5 Derived linear algebraic form	22
14.6 Derived descriptor form	22

15 MNA Equation Builder	23
15.1 build_equations	23
15.2 ResidualSystemBuilder	23
16 Linear Extraction Module	24
16.1 LinearSystem	24
16.2 DescriptorSystem	24
16.3 extract_linear_system	24
16.4 extract_descriptor_system	25
17 Analysis Module	25
17.1 apply_numeric_substitutions	25
17.2 apply_frequency_substitution	25
17.3 collect_parameter_symbols	26
17.4 parse_param_assignment	26
17.5 parse_param_assignments	26
18 Export Functions	26
18.1 export_text	26
18.2 export_json	27
18.3 export_latex	27
18.4 Linear and descriptor text exporters	27
18.5 Linear and descriptor JSON exporters	28
18.6 Linear and descriptor LaTeX exporters	28
19 Error Hierarchy	28
 IV Architecture and Developer Guide	 29
20 Architecture Overview	29
20.1 Architectural principles	29
20.2 Data flow	30
20.3 Module dependency graph	30
21 Adding a New Component	30
22 Testing Guide	31
22.1 Test structure	31
22.2 Mathematical equivalence	32
22.3 ngspice integration tests	32
22.4 Current local health check	32
A Summary of Public API	32
B Reference Circuit Index	33
C Version History	34

Part I

User Guide

1 Introduction

dnipro is a Python 3.11+ library and command-line tool for *symbolic circuit analysis* based on equation-form Modified Nodal Analysis (MNA). Given a SPICE-like netlist, dnipro constructs an algebraic system of residual equations in SymPy symbolic form:

$$F(x(t), \dot{x}(t), t, p) = 0, \quad (1)$$

where $x(t)$ collects node voltages and branch currents, $\dot{x}(t)$ denotes their time derivatives, t is time, and p is the set of symbolic or numeric circuit parameters.

1.1 What dnipro is

dnipro is designed for use cases where the *equations themselves* are the output: teaching circuit theory, generating symbolic transfer functions, preparing derivations for technical reports, or cross-checking hand analysis against a reference tool. The library exports residual equations as human-readable text, JSON (for downstream tooling), or LaTeX (for direct inclusion in documents). For linear systems it can also derive and report the classical algebraic MNA form $Ax = b$ and the time-domain descriptor form $Gx + C\dot{x} = b$.

1.2 What dnipro is not

dnipro is not a SPICE replacement. It does not perform transient numerical integration, does not compute operating points automatically for nonlinear circuits, and does not claim full SPICE netlist compatibility. External ngspice is used only as a verification oracle in integration tests and tutorial comparison artifacts. It is never required by the runtime library.

1.3 Relationship to the specification

The companion document *SPEC_dnipro_v1_symbolic_mna_framework* defines the version 1 requirements. This manual describes the implemented library and CLI as they exist in dnipro 0.1.0.

1.4 Relationship to the User's Guide

This reference manual is organized around the implemented API, syntax, CLI, and export formats. The companion document *Dnipro User's Guide by Examples* is organized around runnable circuits in `examples/user_guide`. Use the User's Guide when learning the system from concrete circuits, dnipro Python scripts, and matching ngspice templates.

2 Installation and Setup

2.1 Requirements

- Python 3.11 or later
- SymPy 1.14 or later (installed automatically as a dependency)
- ngspice — optional; required only for integration tests marked `@pytest.mark.ngspice`
- NumPy — optional; used by selected tutorial scripts for fast numeric AC sweeps and comparison with ngspice output.

2.2 Installing from source

```
git clone <repository>
cd dnipro
pip install -e .
```

After installation the command `dnipro` is available on the `PATH` and the package `dnipro` is importable from Python.

2.3 Verifying the installation

```
dnipro --help
python -c "import dnipro; print(dnipro.__version__)"
```

2.4 Running the test suite

```
pip install pytest
pytest
```

To include ngspice-dependent integration tests:

```
pytest -m ngspice
```

Tests marked `ngspice` are automatically skipped when the `ngspice` binary is absent from `PATH`.

3 Quick Start

3.1 Example 1: Resistive voltage divider (time domain)

Create a netlist file `divider.cir`:

```
* Resistive voltage divider
V1 in 0 Vin
R1 in out R1
R2 out 0 R2
```

Analyze it:

```
dnipro analyze divider.cir
```

Sample output:

```

dnipro residual system
domain: time
nodes: in, out
branch currents: V1
unknowns:
  [0] v_in(t)
  [1] v_out(t)
  [2] i_V1(t)
residuals:
  KCL(in): (v_in(t) - v_out(t))/R1 + i_V1(t) = 0
  KCL(out): -(v_in(t) - v_out(t))/R1 + (v_out(t))/R2 = 0
  V1: v_in(t) - Vin = 0

```

3.2 Example 2: RC low-pass filter (Laplace domain)

```

* RC low-pass filter
V1 in 0 Vin
R1 in out R
C1 out 0 C

```

```
dnipro analyze rc.cir --domain laplace --output latex
```

The output is a $\text{\LaTeX}$ fragment containing the Laplace-domain residuals, suitable for inclusion in a report with `\input{...}`.

3.3 Example 3: Numeric substitution

```

dnipro analyze rc.cir --domain laplace --numeric \
  --param Vin=1 --param R=1k --param C=1u \
  --output json

```

All three symbolic parameters (Vin, R, C) are substituted before export. If any required parameter is missing dnipro prints a clear error and exits with status 1.

3.4 Example 4: Classical linear MNA report

For a linear circuit, request the derived algebraic matrix form:

```
dnipro analyze divider.cir --linear-system
```

Sample output includes row names, the unknown vector, the coefficient matrix, and the right-hand-side vector:

```

dnipro linear MNA system
domain: time
form: A*x = b

row names:
  [0] KCL(in)
  [1] KCL(out)
  [2] V1

unknown vector x:
  [0] v_in(t)

```

```

[1] v_out(t)
[2] i_V1(t)

A matrix:
[1/R1, -1/R1, 1]
[-1/R1, 1/R1 + 1/R2, 0]
[1, 0, 0]

b vector:
[0]
[0]
[Vin]

```

3.5 Example 5: Descriptor MNA report

For a time-domain linear circuit with reactive elements, request descriptor form:

```
dnipro analyze rc.cir --descriptor-system --output latex
```

The output is a $\text{\LaTeX}$ fragment containing the row mapping, unknown and derivative mapping, and displayed G , C , x , $\dot{x}$, and b objects for the equation $Gx + C\dot{x} = b$.

4 Command-Line Interface

4.1 General form

```
dnipro analyze <netlist> [options]
```

<netlist> is the path to a SPICE-like netlist file (UTF-8 encoding).

4.2 Options

Option	Description
--domain {time laplace}	Select the symbolic domain for equation construction. Default: time.
--numeric	Apply numeric substitution to all free parameters. Must be combined with --param for each symbolic parameter present in the circuit. Raises an error if any parameter is missing.
--param NAME=VALUE	Supply or override a parameter value. NAME must be a valid identifier; VALUE follows the same grammar as netlist values (plain number, SPICE suffix, or single symbol). May be repeated.
--freq FREQ_HZ	Substitute $s = j2\pi f$ in Laplace-domain mode. FREQ_HZ is parsed as a netlist value.
--omega OMEGA	Substitute $s = j\omega$ in Laplace-domain mode. OMEGA is parsed as a netlist value.
--output {text json latex}	Select the output format. Default: text.

<code>--linear-system</code>	Export a derived classical linear MNA system $Ax = b$ instead of the residual equations. Available in both time and laplace domains for systems linear in the unknown vector.
<code>--descriptor-system</code>	Export the time-domain descriptor system $Gx + C\dot{x} = b$. Mutually exclusive with <code>--linear-system</code> ; rejected in laplace domain.
<code>--formulas</code>	Legacy alias; ignored in version 0.1.0.

4.3 Exit codes

Code	Meaning
0	Analysis completed successfully; output written to stdout.
1	An error occurred (parse error, missing parameter, file I/O error). The error message is written to stderr.

4.4 Extended examples

```
# Time-domain text output (default)
dnipro analyze circuit.cir

# Laplace domain, JSON output
dnipro analyze circuit.cir --domain laplace --output json

# Frequency substitution at 1 kHz
dnipro analyze filter.cir --domain laplace --freq 1k --output text

# Angular frequency substitution
dnipro analyze filter.cir --domain laplace --omega 6283.18 --output json

# Full numeric substitution
dnipro analyze rc.cir --domain time --numeric \
    --param Vin=5 --param R=1k --param C=100n

# Partial symbolic + LaTeX output
dnipro analyze amp.cir --domain laplace --output latex

# Classical linear MNA matrix/vector report
dnipro analyze amp.cir --domain laplace --linear-system --output latex

# Time-domain descriptor report
dnipro analyze rc.cir --descriptor-system --output text
```

5 Packaged Examples and Tutorials

The repository includes small example circuits and report-generation scripts. These files are not required by the library API, but they are useful for checking the implemented conventions against concrete circuits.

5.1 pub1: small-signal VCCS example

The `pub1/` directory contains a compact small-signal equivalent circuit and ready-made matrix reports:

File	Purpose
<code>pub1/smallsig_full.cir</code>	Full ngspice-style source netlist with <code>.param</code> , <code>.control</code> , and reporting commands. It is intended for ngspice, not direct dnipro parsing.
<code>pub1/smallsig_dnipro_v1.cir</code>	dnipro v1-compatible version of the same circuit. Use this file with <code>dnipro analyze</code> .
<code>pub1/smallsig_mna_matrices.py</code>	Python helper that prints the residual system, $Ax = b$, descriptor form, JSON, and L ^A T _E X fragments.
<code>pub1/smallsig_linear_mna.tex</code>	L ^A T _E X fragment for the classical linear $Ax = b$ report.
<code>pub1/smallsig_descriptor_mna.tex</code>	L ^A T _E X fragment for $Gx + C\dot{x} = b$. For this purely resistive small-signal circuit, C is the zero matrix.
<code>pub1/*_document.tex</code> , <code>pub1/*_document.pdf</code>	Complete wrapper documents and generated PDFs for the two L ^A T _E X fragments.

Useful commands:

```
dnipro analyze pub1/smallsig_dnipro_v1.cir --linear-system
dnipro analyze pub1/smallsig_dnipro_v1.cir --descriptor-system
python pub1/smallsig_mna_matrices.py
```

5.2 MOSFET small-signal tutorial

The `tutorials/` directory contains a MOSFET small-signal equivalent model expressed as explicit linear small-signal elements. This is not a built-in MOSFET device model; it is a circuit-level equivalent built from supported dnipro v1 primitives.

File	Purpose
<code>tutorials/mosfet_small_signal_full.cir</code>	Full ngspice AC analysis netlist with <code>.param</code> , <code>.ac</code> , and <code>.control</code> .
<code>tutorials/mss.out</code>	Captured ngspice output containing AC sweep tables for $v(D)$ and $i(Vin)$.
<code>tutorials/mosfet_small_signal_build.py</code>	Builds dnipro Laplace-domain residual system and classical $Ax = b$ matrix for the same explicit small-signal equivalent. Optional flags print L ^A T _E X or large symbolic transfer expressions.
<code>tutorials/mosfet_small_signal_eval.py</code>	Evaluates dnipro extracted MNA matrix numerically at the same AC sweep frequencies as <code>mss.out</code> and compares $v(D)$ and $i(Vin)$ against ngspice.

Symbolic matrix construction:

```
python tutorials/mosfet_small_signal_dnipro.py
python tutorials/mosfet_small_signal_dnipro.py --latex
python tutorials/mosfet_small_signal_dnipro.py --solve
```

Numerical comparison against ngspice output:

```
python tutorials/mosfet_small_signal_numeric_dnipro.py
python tutorials/mosfet_small_signal_numeric_dnipro.py --all
```

The numeric script substitutes the tutorial parameters, evaluates $s = j2\pi f$, solves $A(f)x = b$ for every frequency point, and reports the absolute difference between dnipro and ngspice results. In the checked-in `mss.out` comparison, the observed maximum absolute differences are on the order of 10^{-5} for $v(D)$ and 10^{-9} for $i(Vin)$, consistent with printed ngspice output precision.

6 Netlist Syntax

6.1 General rules

A netlist is a plain UTF-8 text file. Each non-empty, non-comment line defines one circuit element or model. Lines beginning with `*` are comments and are ignored. Blank lines are ignored. Fields on an element line are separated by whitespace.

6.2 Node names

The following names are ground aliases and are normalized to the internal ground node:

```
0  GND  gnd
```

All other node names must match the identifier pattern `[A-Za-z_][A-Za-z0-9_]*`. Node names are case-sensitive (except the three ground aliases above). The ground node is never included in the unknown vector.

6.3 Component names

A component name must match the pattern `[A-Za-z][A-Za-z0-9_]*`. The first character determines the element type (case-insensitive). The original name is preserved in output. Duplicate component names in the same netlist are rejected.

6.4 Supported element lines

Type	Syntax	Meaning
R	Rname np nm value	Linear resistor.
C	Cname np nm value	Linear capacitor.
L	Lname np nm value	Linear inductor.
V	Vname np nm source	Independent voltage source.
I	Iname np nm source	Independent current source.
E	Ename np nm ncp ncm gain	Voltage-controlled voltage source (VCVS).

G	Gname np nm ncp ncm gm	Voltage-controlled current source (VCCS).
F	Fname np nm Vctrl gain	Current-controlled current source (CCCS).
H	Hname np nm Vctrl rm	Current-controlled voltage source (CCVS).
D	Dname anode cathode model	Diode.
	.model .model name D(Is=... n=... Vt=...)	Diode model definition.

6.5 Values

Version 1 accepts three kinds of values for component parameters:

1. **Plain numbers:** integers, decimals, or scientific notation. Examples: 1, 2.5, 1e-6, -3.3e3.
2. **Numbers with SPICE suffixes** (see Table 6). Examples: 1k, 100n, 25.85m, 4.7u.
3. **Single symbolic identifiers** matching `[A-Za-z_][A-Za-z0-9_]*`. Examples: R, C1val, Vin, gm.

Arbitrary expressions such as `R/2` or `C*(1+alpha)` are intentionally rejected in version 1. Python `eval()` is never used for parsing. Numeric values are represented as exact SymPy Rational objects.

Table 6: SPICE suffix scale factors

Suffix	Scale	Example
f	10^{-15}	1f
p	10^{-12}	1p
n	10^{-9}	100n
u	10^{-6}	1u
m	10^{-3}	25.85m
k	10^3	1k
meg	10^6	1meg
g	10^9	1g
t	10^{12}	1t

The suffix m means milli (10^{-3}). Use meg for mega (10^6). Uppercase M is not recognized in version 1.

6.6 Diode model syntax

```
.model ModelName D(Is=value n=value Vt=value)
```

Exactly the three parameters Is, n, and Vt must be present (space-separated, not comma-separated). Each value follows the standard value grammar. Duplicate model names in the same netlist are rejected.

7 Source Functions

Independent voltage (V) and current (I) sources accept a source specification as their last field(s). All source functions are converted to SymPy expressions of time t during parsing.

7.1 DC (constant)

```
V1 in 0 value
V1 in 0 DC(value)
```

Generates the constant expression $f(t) = value$.

7.2 SIN (sinusoidal)

```
V1 in 0 SIN(offset amplitude omega)
```

Generates:

$$f(t) = offset + amplitude \cdot \sin(\omega t). \quad (2)$$

All three arguments are required.

7.3 PULSE

```
V1 in 0 PULSE(v1 v2 delay rise_time fall_time width period)
```

All seven arguments are required (space-separated). Generates a periodic, trapezoid-shaped waveform using SymPy Piecewise and floor. Closed-form analytical solution of circuits driven by PULSE sources is not required in version 1.

The periodic phase variable is:

$$\tau = t - delay - period \left\lfloor \frac{t - delay}{period} \right\rfloor. \quad (3)$$

7.4 PWL (piecewise linear)

```
V1 in 0 PWL(t0 v0 t1 v1 t2 v2 ...)
```

At least two time/value pairs are required. Arguments must be given in pairs (even count); values are space-separated. Generates a SymPy Piecewise linear interpolation between the provided points. The source holds v_0 for $t \leq t_0$ and v_{n-1} for $t > t_{n-1}$.

7.5 SQUARE (square wave)

```
V1 in 0 SQUARE(offset amplitude omega duty)
```

All four arguments are required. *duty* is the duty cycle fraction in $(0, 1)$. Generates a periodic square wave using a normalized phase variable:

$$\phi = \frac{\omega t}{2\pi} - \left\lfloor \frac{\omega t}{2\pi} \right\rfloor \in [0, 1). \quad (4)$$

$$f(t) = \begin{cases} \text{offset} + \text{amplitude} & \phi < \text{duty}, \\ \text{offset} - \text{amplitude} & \text{otherwise.} \end{cases} \quad (5)$$

7.6 SAW (sawtooth)

V1 in 0 **SAW**(offset amplitude period)

All three arguments are required. Generates a periodic linear ramp:

$$\tau = t - \text{period} \left\lfloor \frac{t}{\text{period}} \right\rfloor, \quad f(t) = \text{offset} + \text{amplitude} \frac{\tau}{\text{period}}. \quad (6)$$

Part II

Mathematical Reference

8 MNA Theory

8.1 Overview of Modified Nodal Analysis

Modified Nodal Analysis is a systematic method for formulating circuit equations. The standard MNA matrix form is

$$Gx(t) + C\dot{x}(t) = b(t), \quad (7)$$

where G is the conductance matrix, C is the reactive (capacitive/inductive) matrix, and $b(t)$ collects source contributions.

dnipro does not use the matrix form as its primary representation. Instead it constructs the equivalent *residual form*:

$$F(x(t), \dot{x}(t), t, p) = 0. \quad (8)$$

The residual form is more general: it accommodates nonlinear components (such as diodes) in the same framework without restructuring the solver. Linear matrix forms are special cases of the residual form.

8.2 Unknown vector

For a circuit with N non-ground nodes and B voltage-defined branches the unknown vector is:

$$x(t) = \begin{bmatrix} v_{n_1}(t) \\ \vdots \\ v_{n_N}(t) \\ i_{b_1}(t) \\ \vdots \\ i_{b_B}(t) \end{bmatrix}. \quad (9)$$

In *time-domain* mode, node voltages and branch currents are SymPy Function objects applied to the time symbol t , e.g. $v_{\text{out}}(t)$ and $i_{V1}(t)$. This enables SymPy `diff()` to generate symbolic derivatives.

In *Laplace-domain* mode, they are plain SymPy Symbol objects e.g. v_{out} and i_{V1} , with the complex frequency $s = \text{Symbol}("s")$.

The ordering is deterministic:

1. Non-ground node voltages in first-appearance order.
2. Branch current unknowns in first-appearance order of their components.

8.3 Residual vector

The residual vector F has the same length as x . Its entries are ordered as:

1. One KCL residual per non-ground node, in node order.
2. One branch constraint residual per voltage-defined branch, in component order.

8.4 KCL residuals

For each non-ground node n , the KCL residual is formed as the algebraic sum of currents *leaving* the node:

$$F_n = \sum_k i_k^{(\text{out})}(n) = 0. \quad (10)$$

Here, $i_k^{(\text{out})}(n)$ is positive when current flows away from node n through branch k .

Internally, the builder maintains one accumulator per non-ground node. The operation `add_current(np, nm, current)` adds current to node np and subtracts it from node nm . If either terminal is the ground node, no residual is created for ground.

8.5 Voltage-defined branches

The following elements introduce a branch-current unknown and a branch constraint residual:

Element	Branch constraint
Voltage source V	$v(np) - v(nm) - V_s = 0$
VCVS E	$v(np) - v(nm) - \mu[v(ncp) - v(ncm)] = 0$
CCVS H	$v(np) - v(nm) - r_m i_{ctrl} = 0$
Inductor L	$v(np) - v(nm) - L \dot{i}_L = 0$ (time) $v(np) - v(nm) - sL i_L = 0$ (Laplace)

VCCS (G), CCCS (F), resistor (R), capacitor (C), current source (I), and diode (D) do *not* introduce branch-current unknowns.

8.6 Reference direction conventions

For any two-terminal element written as `Xname np nm`:

$$v_X = v(np) - v(nm), \quad i_X : np \rightarrow nm \text{ (positive direction)}. \quad (11)$$

For independent sources:

- A voltage source constrains $v(np) - v(nm) = V_s$; its branch current flows from np to nm .
- A current source injects current *from* np *to* nm through the external circuit.

9 Component Equations

9.1 Resistor

For R1 np nm R, the current from np to nm is:

$$i_R(t) = \frac{v(np, t) - v(nm, t)}{R}. \quad (12)$$

No branch-current unknown is allocated. KCL contribution via `add_current(np, nm, i_R)`.

9.2 Capacitor

Time domain. For C1 np nm C:

$$i_C(t) = C \frac{d}{dt} [v(np, t) - v(nm, t)]. \quad (13)$$

Implemented as `C * diff(voltage, t)`.

Laplace domain. The admittance representation is used:

$$i_C = sC [v(np) - v(nm)]. \quad (14)$$

No branch-current unknown in either domain.

9.3 Inductor

An inductor is treated as a voltage-defined branch in both domains.

Time domain. For L1 np nm L: Branch current $i_{L1}(t)$ is allocated.

$$\text{KCL : } i_{L1}(t) \text{ added from np to nm,} \quad (15)$$

$$\text{Constraint : } v(np, t) - v(nm, t) - L \frac{di_{L1}(t)}{dt} = 0. \quad (16)$$

Laplace domain.

$$\text{KCL : } i_{L1} \text{ added from np to nm,} \quad (17)$$

$$\text{Constraint : } v(np) - v(nm) - sL i_{L1} = 0. \quad (18)$$

9.4 Voltage source

For V1 np nm source: Branch current $i_{V1}(t)$ is allocated.

$$\text{KCL : } i_{V1}(t) \text{ added from np to nm,} \quad (19)$$

$$\text{Constraint : } v(np, t) - v(nm, t) - f(t) = 0, \quad (20)$$

where $f(t)$ is the source expression (DC, SIN, etc.).

9.5 Current source

For I1 np nm source: The source expression $f(t)$ is added directly to KCL as current from np to nm. No branch-current unknown is allocated.

9.6 VCVS (E source)

For E1 np nm ncp ncm mu: Branch current $i_{E1}(t)$ is allocated.

$$\text{KCL: } i_{E1}(t) \text{ added from np to nm,} \quad (21)$$

$$\text{Constraint: } v(np) - v(nm) - \mu[v(ncp) - v(ncm)] = 0. \quad (22)$$

9.7 VCCS (G source)

For G1 np nm ncp ncm gm: The output current from np to nm is:

$$i_G = g_m [v(ncp) - v(ncm)]. \quad (23)$$

No branch-current unknown is allocated.

9.8 CCCS (F source)

For F1 np nm Vctrl beta: The output current from np to nm is:

$$i_F = \beta i_{Vctrl}. \quad (24)$$

Vctrl must be the name of an existing voltage-defined branch. No branch-current unknown is allocated for the F element itself.

9.9 CCVS (H source)

For H1 np nm Vctrl rm: Branch current i_{H1} is allocated.

$$\text{KCL: } i_{H1} \text{ added from np to nm,} \quad (25)$$

$$\text{Constraint: } v(np) - v(nm) - r_m i_{Vctrl} = 0. \quad (26)$$

Vctrl must be the name of an existing voltage-defined branch.

9.10 Diode

For D1 anode cathode Model: The diode voltage is $v_D = v(\text{anode}) - v(\text{cathode})$. The diode current from anode to cathode is the Shockley equation:

$$i_D = I_S \left(\exp\left(\frac{v_D}{n V_T}\right) - 1 \right), \quad (27)$$

where I_S , n , V_T come from the named .model definition. The current i_D is added to KCL via `add_current(anode, cathode,  $i_D$ )`. No branch-current unknown is allocated.

10 Time Domain and Laplace Domain

10.1 Time-domain mode

Selected with `--domain time` (the default).

Node voltages and branch currents are SymPy Function objects evaluated at $t = \text{Symbol("t", real=True)}$. For example, a node out gives the unknown $v_{\text{out}}(t)$. Time derivatives are formed with `sympy.diff(expr, t)`.

Capacitor currents take the form $C \dot{v}(t)$; inductor constraints include $L \dot{i}_L(t)$.

10.2 Laplace-domain mode

Selected with `--domain laplace`.

Node voltages and branch currents are plain SymPy Symbol objects. The complex frequency is $s = \text{Symbol}("s")$. Capacitors contribute $sC V$ and inductors use the constraint $V - sL I = 0$.

10.3 Frequency substitution

Frequency substitution replaces s with a concrete complex value. It is only available in Laplace-domain mode.

Option	Substitution
<code>--freq F</code>	$s \leftarrow j 2\pi f$
<code>--omega W</code>	$s \leftarrow j \omega$

After substitution, residuals are complex-valued SymPy expressions. The substitution is recorded in `ResidualSystem.metadata["frequency_substitution"]`.

11 Nonlinear Component: Diode

11.1 Design restriction

Version 1 supports exactly one nonlinear component: the junction diode. The diode is handled inside the same residual system as all linear elements. There is no separate nonlinear backend.

11.2 What is not implemented

The following are explicitly out of scope for version 1:

- Jacobian matrix construction or export.
- Newton-Raphson or any other nonlinear solver.
- Automatic small-signal linearization.
- BJT or MOSFET models.

If `--numeric` is applied to a circuit containing a diode, dnipro substitutes numerical parameter values into the symbolic residuals but does *not* attempt to solve the resulting nonlinear system.

11.3 Model parameters

The diode model requires exactly three parameters:

Parameter	Symbol	Meaning
<code>Is</code>	I_S	Saturation current (A)
<code>n</code>	n	Emission coefficient (dimensionless)
<code>Vt</code>	V_T	Thermal voltage (V); typically ≈ 25.85 mV

11.4 Example

```
V1 in 0 Vin
R1 in out R
D1 out 0 Dmod
.model Dmod D(Is=1e-14 n=1 Vt=25.85m)
```

The residual system (time domain) will contain:

$$F_{\text{in}} = \frac{v_{\text{in}}(t) - v_{\text{out}}(t)}{R} + i_{V1}(t) = 0, \quad (28)$$

$$F_{\text{out}} = \frac{v_{\text{out}}(t) - v_{\text{in}}(t)}{R} + I_S \left(\exp\left(\frac{v_{\text{out}}(t)}{n V_T}\right) - 1 \right) = 0, \quad (29)$$

$$F_{V1} = v_{\text{in}}(t) - V_{\text{in}} = 0. \quad (30)$$

Part III

Library API Reference

12 Package Overview

12.1 Public API

The top-level package dnipro exports three convenience names:

```
from dnipro import parse_netlist_file, parse_netlist_text, build_equations
import dnipro
print(dnipro.__version__) # "0.1.0"
```

Additional public module-level APIs are documented in the sections below. Symbols not documented in this manual should be treated as internal.

12.2 Package structure

```
dnipro/
  __init__.py          # public exports
  errors.py            # exception hierarchy
  parser/
    netlist_parser.py  # parse_netlist_text, parse_netlist_file
    value_parser.py    # parse_value
    source_parser.py   # parse_source, source function classes
  circuit/
    circuit.py         # Circuit
    components.py      # Component subclasses
    models.py          # DiodeModel
    nodes.py           # normalize_node, GROUND
  mna/
    equation_builder.py # build_equations, ResidualSystemBuilder
    linear_extraction.py # LinearSystem, DescriptorSystem, extraction
  functions
    residual_system.py  # ResidualSystem
  analysis/
    substitutions.py    # substitution functions
```

```
export/  
  text_exporter.py      # residual, linear, and descriptor text exporters  
  json_exporter.py      # residual, linear, and descriptor JSON exporters  
  latex_exporter.py     # residual, linear, and descriptor LaTeX exporters  
cli/  
  main.py               # main()
```

13 Parser Module

13.1 parse_netlist_text

`parse_netlist_text(text: str) -> Circuit`

Module: `dnipro.parser.netlist_parser` (also exported from `dnipro`)

Parameters:

- `text` — the netlist as a string.

Returns: A `Circuit` object.

Raises:

- `ParseError` — on malformed element lines, invalid node or component names, duplicate component names.
- `UnsupportedElementError` — on unsupported element type letters or unsupported directives.
- `UnsupportedValueError` — on disallowed value syntax (e.g. expressions, comma-separated source arguments).
- `ModelError` — on malformed `.model` lines, unsupported model types, missing or extra model parameters, duplicate model names.

Comment lines (starting with `*`) and blank lines are silently ignored.

13.2 parse_netlist_file

`parse_netlist_file(path: str) -> Circuit`

Module: `dnipro.parser.netlist_parser` (also exported from `dnipro`)

Parameters:

- `path` — filesystem path to the netlist file (UTF-8).

Returns: A `Circuit` object.

Raises: Same exceptions as `parse_netlist_text`; also `OSError` on file I/O failure.

Thin wrapper around `parse_netlist_text`: reads the file and delegates.

13.3 parse_value

`parse_value(raw: str) -> sympy.Expr`

Module: `dnipro.parser.value_parser`

Parameters:

- `raw` — a value string; leading/trailing whitespace is stripped.

Returns: A SymPy expression: `Rational` for numeric values, `Symbol` for identifier values.

Raises: `UnsupportedValueError` on empty input or disallowed syntax.

Numeric values are represented as exact SymPy `Rational` objects (no floating-point rounding).

13.4 parse_source

`parse_source(raw: str) -> SourceFunction`

Module: `dnipro.parser.source_parser`

Parameters:

- `raw` — source specification string, e.g. `"5"`, `"SIN(0 Vm omega)"`, `"PULSE(0 1 0 1n 1n 500n 1u)"`.

Returns: A frozen dataclass implementing the `SourceFunction` protocol. Call `.to_sympy(t)` to obtain the SymPy expression.

Raises: `UnsupportedValueError` on unsupported function names, wrong argument counts, or comma-separated arguments.

The returned object's `.kind` attribute is one of `"DC"`, `"SIN"`, `"PULSE"`, `"PWL"`, `"SQUARE"`, `"SAW"`.

14 Data Model

14.1 Circuit

```
@dataclass
class Circuit:
    components: list[Component]
    node_order: list[str]
    component_names: set[str]
    models: dict[str, DiodeModel]
```

Module: `dnipro.circuit.circuit`

Mutable container built incrementally by the parser. The `node_order` list preserves first-appearance order (excluding ground).

Methods:

`add_component(component: Component) -> None`

Appends component to `components`, records its name in `component_names`, and extends `node_order` with any new nodes. Raises `ParseError` on a duplicate component name.

`add_model(model: DiodeModel) -> None`

Adds model to `models` keyed by `model.name`. Raises `ModelError` on a duplicate model name.

14.2 Component classes

All component classes are frozen dataclasses in `dnipro.circuit.components`. The base class is:

```
@dataclass(frozen=True)
class Component:
    name: str # component name (preserved from netlist)
    np: str # positive node (normalized)
    nm: str # negative node (normalized)
```

Subclasses and their additional fields:

Class	Additional fields
Resistor	value: <code>sympy.Expr</code>
Capacitor	value: <code>sympy.Expr</code>
Inductor	value: <code>sympy.Expr</code>
VoltageSource	source_value: <code>sympy.Expr</code> (already converted to SymPy via <code>parse_source</code>)
CurrentSource	source_value: <code>sympy.Expr</code>
Diode	model_name: <code>str</code>
VCVS	ncp: <code>str</code> , ncm: <code>str</code> , gain: <code>sympy.Expr</code>
VCCS	ncp: <code>str</code> , ncm: <code>str</code> , transconductance: <code>sympy.Expr</code>
CCCS	control_branch: <code>str</code> , gain: <code>sympy.Expr</code>
CCVS	control_branch: <code>str</code> , transresistance: <code>sympy.Expr</code>

The fields `ncp` and `ncm` of `VCVS` and `VCCS` are the positive and negative control nodes (normalized). The field `control_branch` of `CCCS` and `CCVS` is the name of the voltage-defined branch whose current is sensed.

14.3 DiodeModel

```
@dataclass(frozen=True)
class DiodeModel:
    name: str
    saturation_current: sympy.Expr
    emission_coefficient: sympy.Expr
    thermal_voltage: sympy.Expr
```

Module: `dnipro.circuit.models`

14.4 ResidualSystem

```
@dataclass(frozen=True)
class ResidualSystem:
    domain: str
    unknowns: list[sympy.Expr]
    residuals: list[sympy.Expr]
    node_order: list[str]
    branch_current_order: list[str]
    residual_names: list[str]
    metadata: dict
```

Module: `dnipro.mna.residual_system`

A frozen, immutable value object. Analysis functions (`apply_numeric_substitutions`, `apply_frequency_substitution`) do not modify this object; they return a new `ResidualSystem`.

Fields:

- `domain` — "time" or "laplace".
- `unknowns` — ordered list of SymPy expressions, length $N+B$ (nodes then branch currents).
- `residuals` — ordered list of SymPy residual expressions, same length as `unknowns`.
- `node_order` — names of non-ground nodes in first-appearance order.
- `branch_current_order` — names of voltage-defined branches in first-appearance order.
- `residual_names` — human-readable labels for each residual, e.g. "KCL(out)", "V1", "L1".
- `metadata` — dictionary recording analysis history. Keys used by the library: "substitutions" (from numeric substitution) and "frequency_substitution" (from frequency substitution).

The i -th unknown corresponds to the i -th residual.

14.5 Derived linear algebraic form

For residual systems that are linear in the unknown vector, dnipro can derive the classical matrix equation

$$Ax = b. \quad (31)$$

The unknown vector x is exactly `ResidualSystem.unknowns`; no reordering is performed during extraction. The row order of A and b is exactly the order of `ResidualSystem.residuals` and row labels are preserved through `ResidualSystem.residual_names`.

The extraction uses SymPy linear-equation utilities after replacing time-domain unknown functions such as $v_{\text{out}}(t)$ with temporary symbols. This makes the operation compatible with both time-domain and Laplace-domain dnipro residual systems.

If a residual contains a nonlinear dependence on an unknown, for example the exponential current of a diode, extraction raises `NonlinearSystemError`.

14.6 Derived descriptor form

For time-domain linear systems, dnipro can also derive

$$Gx(t) + C\dot{x}(t) = b(t), \quad (32)$$

where

$$\dot{x}(t) = \left[\frac{d}{dt}x_0(t) \quad \cdots \quad \frac{d}{dt}x_{n-1}(t) \right]^T. \quad (33)$$

The derivative vector preserves the same order as x . Capacitor terms contribute to C through voltage derivatives, and time-domain inductor branch constraints contribute through branch-current derivatives. Descriptor extraction is intentionally rejected for Laplace-domain systems because Laplace residuals are already algebraic in s .

15 MNA Equation Builder

15.1 build_equations

`build_equations(circuit: Circuit, domain: str = "time") -> ResidualSystem`

Module: `dnipro.mna.equation_builder` (also exported from `dnipro`)

Parameters:

- `circuit` — a `Circuit` object returned by the parser.
- `domain` — "time" (default) or "laplace".

Returns: A `ResidualSystem`.

Raises:

- `UnsupportedElementError` — on an unsupported domain string or an unrecognized component type.
- `ControlBranchError` — when a CCCS or CCVS references a control branch that is not a voltage-defined branch (i.e. the named component does not appear in `branch_current_order`).
- `ModelError` — when a Diode component references a model name not present in `circuit.models`.

This is the primary entry point for equation construction. Internally it creates a `ResidualSystemBuilder` and calls its `build()` method.

15.2 ResidualSystemBuilder

```
@dataclass
class ResidualSystemBuilder:
    circuit: Circuit
    domain: str
    time: sympy.Symbol # Symbol("t", real=True)
    frequency: sympy.Symbol # Symbol("s")
```

Module: `dnipro.mna.equation_builder`

Lower-level class used internally by `build_equations`. Available for advanced use when components need to be added programmatically.

Methods:

`node_voltage(node_name: str) -> sympy.Expr`

Returns `Integer(0)` for the ground node. Returns a `Function` of `self.time` in time-domain mode, or a `Symbol` in Laplace-domain mode.

`branch_current(component_name: str) -> sympy.Expr`

Returns or creates the branch-current unknown for the named component. Creates a `Function` of `self.time` or a `Symbol` depending on `self.domain`.

`add_current(np: str, nm: str, current_expr: sympy.Expr) -> None`

Adds `current_expr` to the KCL accumulator of node `np` and subtracts it from node `nm`. Ground nodes are silently skipped.

`add_constraint(name: str, expr: sympy.Expr) -> None`

Appends a branch constraint residual `expr = 0` labelled `name`.

`build()` -> `ResidualSystem`

Processes all components in the circuit and returns the completed `ResidualSystem`.

16 Linear Extraction Module

All functions are in `dnipro.mna.linear_extraction`. They operate on an already-built `ResidualSystem` and return derived matrix views; the residual system remains the primary representation.

16.1 LinearSystem

```
@dataclass(frozen=True)
class LinearSystem:
    matrix: sympy.Matrix
    unknown_vector: sympy.Matrix
    rhs: sympy.Matrix
    residual_names: list[str]
    metadata: dict
```

Module: `dnipro.mna.linear_extraction`

Represents the derived algebraic system $Ax = b$. `matrix` is A , `unknown_vector` is x , and `rhs` is b . The metadata dictionary is copied from the source residual system and includes "domain".

16.2 DescriptorSystem

```
@dataclass(frozen=True)
class DescriptorSystem:
    conductance_matrix: sympy.Matrix
    dynamic_matrix: sympy.Matrix
    unknown_vector: sympy.Matrix
    derivative_vector: sympy.Matrix
    rhs: sympy.Matrix
    residual_names: list[str]
    metadata: dict
```

Module: `dnipro.mna.linear_extraction`

Represents the derived descriptor system $Gx + C\dot{x} = b$. `conductance_matrix` is G , `dynamic_matrix` is C , and `derivative_vector` is $\dot{x}$. Convenience aliases `DescriptorSystem.G` and `DescriptorSystem.C` return the same matrices.

16.3 extract_linear_system

`extract_linear_system(system: ResidualSystem) -> LinearSystem`

Parameters:

- `system` — a residual system from `build_equations` or an analysis function.

Returns: A `LinearSystem` preserving the source unknown order and residual row labels.

Raises: `NonlinearSystemError` if any residual is nonlinear in the unknown vector.

16.4 extract_descriptor_system

`extract_descriptor_system(system: ResidualSystem) -> DescriptorSystem`

Parameters:

- `system` — a time-domain residual system.

Returns: A `DescriptorSystem` preserving source unknown order, derivative order, and residual row labels.

Raises:

- `NonlinearSystemError` if the system is not time-domain.
- `NonlinearSystemError` if any residual is nonlinear in x or $\dot{x}$.

17 Analysis Module

All functions are in `dnipro.analysis.substitutions`. None of them modify their input; each returns a new `ResidualSystem`.

17.1 apply_numeric_substitutions

```
apply_numeric_substitutions(
    system: ResidualSystem,
    substitutions: Mapping[sympy.Symbol, sympy.Expr],
) -> ResidualSystem
```

Parameters:

- `system` — the input residual system.
- `substitutions` — a mapping from SymPy `Symbol` objects to replacement SymPy expressions. Typically built from CLI `--param` arguments via `parse_param_assignments`.

Returns: A new `ResidualSystem` with residuals simplified after substitution. The `metadata["substitutions"]` key records the applied mapping.

Raises: `MissingParameterError` if any free symbol (other than `t` and `s`) in the system is absent from substitutions.

Uses `sympy.simplify` after substitution. The `unknowns` list is not modified.

17.2 apply_frequency_substitution

```
apply_frequency_substitution(
    system: ResidualSystem,
    *,
    freq_hz: sympy.Expr | None = None,
    omega: sympy.Expr | None = None,
) -> ResidualSystem
```

Parameters:

- `system` — must have `domain == "laplace"`.
- `freq_hz` — frequency in Hz; substitutes $s \leftarrow j 2\pi f$.
- `omega` — angular frequency; substitutes $s \leftarrow j\omega$.

Exactly one of `freq_hz` or `omega` must be provided.

Returns: A new `ResidualSystem` with `s` replaced. Records `metadata["frequency_substitution"]` with keys "mode", "value", and "s" (the substituted value).

Raises: `UnsupportedValueError` if the domain is not "laplace" or if both or neither of `freq_hz/omega` are given.

17.3 collect_parameter_symbols

```
collect_parameter_symbols(system: ResidualSystem) -> set[sympy.Symbol]
```

Returns: All free SymPy symbols found in `system.residuals` and `system.unknowns`, excluding the reserved independent variables `t` (`Symbol("t", real=True)`) and `s` (`Symbol("s")`).

Useful for determining which `--param` values are required before calling `apply_numeric_substitution`.

17.4 parse_param_assignment

```
parse_param_assignment(raw: str) -> tuple[sympy.Symbol, sympy.Expr]
```

Parameters:

- `raw` — a string of the form "NAME=VALUE".

Returns: A tuple (`Symbol(NAME)`, `parse_value(VALUE)`).

Raises: `UnsupportedValueError` on missing `=`, invalid name, empty value.

17.5 parse_param_assignments

```
parse_param_assignments(
    raw_assignments: Iterable[str],
) -> dict[sympy.Symbol, sympy.Expr]
```

Parameters:

- `raw_assignments` — iterable of "NAME=VALUE" strings (e.g. the list of `--param` arguments from the CLI).

Returns: A dictionary mapping each `Symbol` to its replacement value.

Raises: `UnsupportedValueError` on malformed assignments or duplicates.

18 Export Functions

18.1 export_text

```
export_text(system: ResidualSystem) -> str
```

Module: `dnipro.export.text_exporter`

Returns a human-readable multi-line string. The format is:

```
dnipro residual system
domain: <domain>
nodes: <n1>, <n2>, ...
branch currents: <c1>, <c2>, ...
unknowns:
```

```

[0] <expr>
...
residuals:
  <name>: <expr> = 0
...
metadata:
  <key>: <value>
...

```

Expressions are serialized with `sympy.sstr()` for stable output. The metadata block is omitted when the metadata dictionary is empty.

18.2 export_json

`export_json(system: ResidualSystem) -> str`

Module: `dnipro.export.json_exporter`

Returns a JSON string with 2-space indentation. All SymPy expressions are serialized as strings via `sympy.sstr()`.

Top-level keys (in output order):

Key	Value
project	"dnipro"
domain	"time" or "laplace"
unknowns	list of expression strings
residual_names	list of label strings
residuals	list of expression strings
node_order	list of node name strings
branch_current_order	list of branch name strings
metadata	nested object; SymPy values serialized to strings

18.3 export_latex

`export_latex(system: ResidualSystem) -> str`

Module: `dnipro.export.latex_exporter`

Returns a $\text{\LaTeX}$ source fragment (not a complete document). Uses `sympy.latex()` for mathematical notation.

The fragment contains:

- `\section*{dnipro residual system}` with domain.
- `\subsection*{Unknowns}` with an `align*` environment listing $x_i = \dots$
- `\subsection*{Residual Equations}` with an `align*` environment listing $F_i(\textit{name}) = \dots = 0$.

Include it in a document with `\input{output.tex}` or paste inline.

18.4 Linear and descriptor text exporters

`export_linear_text(system: LinearSystem) -> str`

`export_descriptor_text(system: DescriptorSystem) -> str`

Module: `dnipro.export.text_exporter`

Return human-readable reports for $Ax = b$ and $Gx + C\dot{x} = b$. Both reports include the equation form, row-name mapping, unknown vector, matrix or matrices, and right-hand-side vector. Descriptor reports additionally include the derivative vector.

18.5 Linear and descriptor JSON exporters

```
export_linear_json(system: LinearSystem) -> str
export_descriptor_json(system: DescriptorSystem) -> str
```

Module: `dnipro.export.json_exporter`

Return JSON strings with stable SymPy `sstr()` serialization. Linear reports use keys "form", "unknowns", "residual_names", "matrix", "rhs", and "metadata". Descriptor reports use "conductance_matrix", "dynamic_matrix", and "derivatives" in addition to the common keys.

18.6 Linear and descriptor LaTeX exporters

```
export_linear_latex(system: LinearSystem) -> str
export_descriptor_latex(system: DescriptorSystem) -> str
```

Module: `dnipro.export.latex_exporter`

Return $\text{\LaTeX}$ source fragments using `sympy.latex()` for matrix and vector entries. Linear output contains row and unknown mapping tables plus displayed A , x , and b . Descriptor output contains row mapping and unknown/derivative mapping tables plus displayed G , C , x , $\dot{x}$, and b .

19 Error Hierarchy

All dnipro-specific exceptions inherit from `DniproError`. Catching `DniproError` covers all library errors.

```
DniproError(Exception)
  ParseError(DniproError)
    UnsupportedElementError(ParseError)
    UnsupportedValueError(ParseError)
  ModelError(DniproError)
  ControlBranchError(DniproError)
  MissingParameterError(DniproError)
  NonlinearSystemError(DniproError)
```

Exception	When raised
<code>DniproError</code>	Base class; never raised directly.
<code>ParseError</code>	Malformed netlist line, invalid name, duplicate component.
<code>UnsupportedElementError</code>	Unknown element type letter or unsupported directive.
<code>UnsupportedValueError</code>	Disallowed value syntax, unsupported source function, comma-separated arguments, wrong argument count.

ModelError	Malformed <code>.model</code> line, unsupported model type, missing/extra model parameters, duplicate model name.
ControlBranchError	CCCS or CVCVS control branch is not a voltage-defined branch.
MissingParameterError	<code>--numeric</code> requested but one or more parameters are absent. <code>exc.missing</code> is a <code>tuple[str, ...]</code> of missing symbol names.
NonlinearSystemError	Linear or descriptor extraction requested for a system that is nonlinear in the selected unknowns, or descriptor extraction requested outside time-domain mode.

Part IV

Architecture and Developer Guide

20 Architecture Overview

20.1 Architectural principles

- The equation builder is the core of the system; the CLI is a thin wrapper.
- Parsers produce typed, immutable dataclass objects, not raw strings.
- Component contribution is implemented via direct method calls on the builder, not through virtual matrix stamps.
- The primary internal representation is the residual system, not a coefficient matrix.
- No feature uses `Python eval()` on user-provided input.
- `ngspice` is used only in external verification tests, never at runtime.

20.2 Data flow

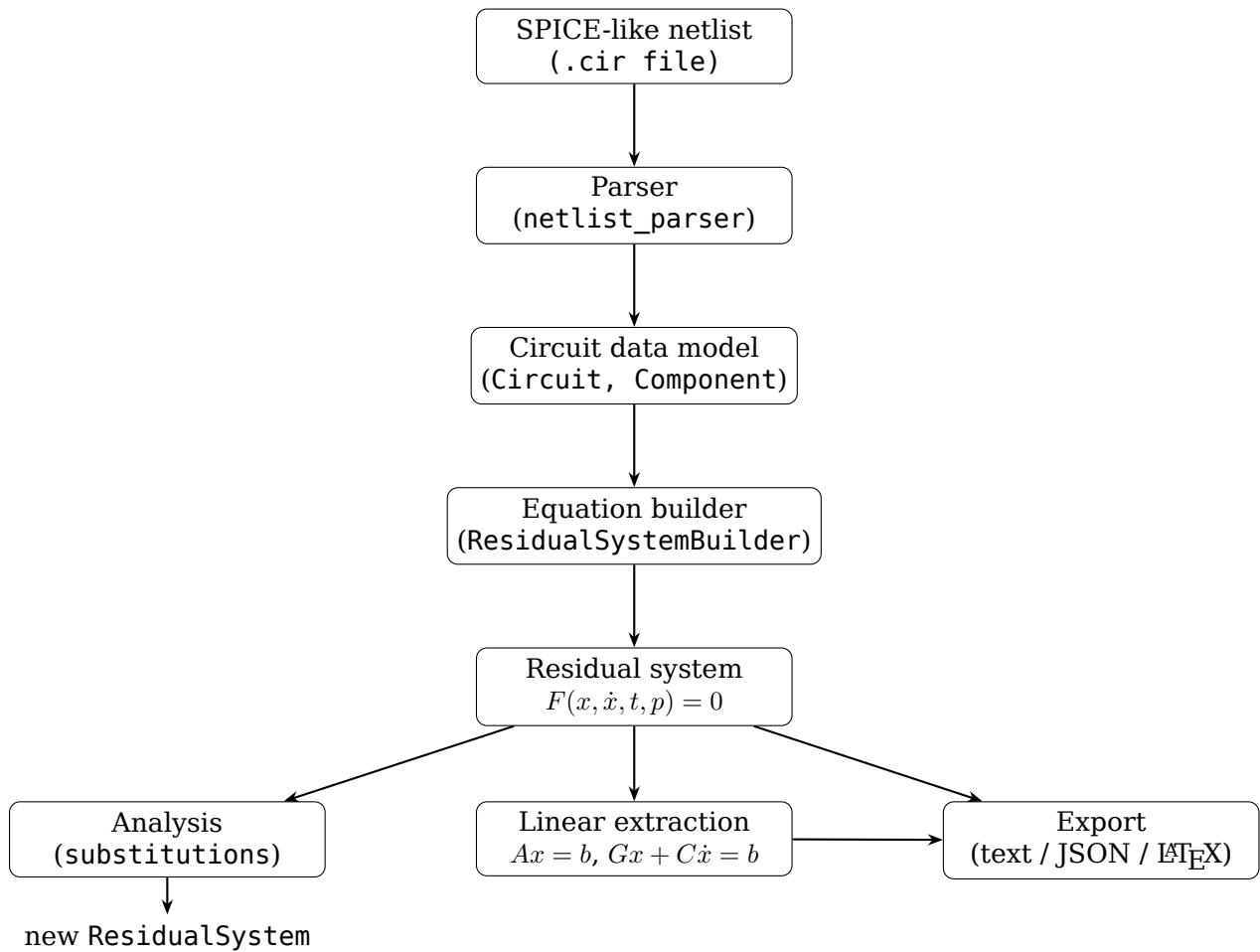


Figure 1: dnipro data flow.

20.3 Module dependency graph

Module	Imports from
cli.main	parser, mna, analysis, export
mna.equation_builder	circuit.*, mna.residual_system, errors
mna.linear_extraction	mna.residual_system, errors
parser.netlist_parser	circuit.*, parser.value_parser, parser.source_parser, errors
analysis.substitutions	mna.residual_system, parser.value_parser, errors
export.*	mna.residual_system, mna.linear_extraction
circuit.*	errors
errors	(none)

21 Adding a New Component

To add a new two-terminal linear component X to version 1:

1. **Add a frozen dataclass** in dnipro/circuit/components.py:

```
@dataclass(frozen=True)
class XElement(Component):
    value: sp.Expr
```

2. **Add a parser branch** in `dnipro/parser/netlist_parser.py`: Add an `if element_type == "X":` branch in `_parse_element_line` and a helper `_parse_xelement` function following the pattern of `_parse_resistor`.
3. **Implement the contribution method** in `dnipro/mna/equation_builder.py`: Add `_contribute_xelement(self, component: XElement)` following the equations for the component. Use `self.add_current(np, nm, expr)` for KCL contributions and `self.add_constraint(name, expr)` for branch constraints.
4. **Dispatch in build()**: Add an `elif isinstance(component, XElement):` branch in the `build()` method to call `_contribute_xelement`.
5. **Write tests** covering:
 - Parser: valid line, field-count errors, invalid values.
 - MNA builder: correct KCL residuals and constraints in both domains.
 - Use `sympy.simplify(actual - expected) == 0` for equation checks.

Voltage-defined branches additionally require: pre-allocating the branch current in the first pass of `build()` by adding `XElement` to the `isinstance` check:

```
if isinstance(component, (VoltageSource, VCVS, C CVS, Inductor, XElement)):
    self.branch_current(component.name)
```

22 Testing Guide

22.1 Test structure

```
tests/
  parser/
    test_value_parser.py
    test_source_parser.py
    test_netlist_parser_rvi.py
    test_netlist_parser_cl.py
    test_netlist_parser_dependent_sources.py
    test_netlist_parser_diode.py
  mna/
    test_equation_builder_rvi.py
    test_equation_builder_cl.py
    test_equation_builder_dependent_sources.py
    test_equation_builder_diode.py
    test_equation_builder_time_sources.py
    test_equation_builder_laplace.py
    test_linear_extraction.py
    test_reference_circuit_coverage.py
  analysis/
    test_substitutions.py
  export/
    test_exporters.py
    test_linear_system_exporters.py
  circuit/
    test_nodes.py
```

```
cli/
  test_main.py
integration/
  test_ngspice_verification.py
```

22.2 Mathematical equivalence

Tests compare symbolic expressions using SymPy simplification:

```
import sympy as sp

def assert_equivalent(actual: sp.Expr, expected: sp.Expr) -> None:
    assert sp.simplify(actual - expected) == 0
```

Output-format tests (JSON, text, LaTeX) may use exact string matching where stable formatting is intentionally verified.

22.3 ngspice integration tests

Tests in tests/integration/ are decorated with `@pytest.mark.ngspice` and skipped when ngspice is absent:

```
import shutil, pytest

@pytest.mark.ngspice
@pytest.mark.skipif(
    shutil.which("ngspice") is None,
    reason="ngspice is not installed"
)
def test_dc_divider():
    ...
```

Run them explicitly with `pytest -m ngspice`.

22.4 Current local health check

At the time this manual was updated, the project health check used:

```
/home/konst/Documents/Projects/venv/bin/python -m pytest -q
/home/konst/Documents/Projects/venv/bin/ruff check .
/home/konst/Documents/Projects/venv/bin/python -m pytest --collect-only -q
```

The collected test count is 165 tests. The full test suite and Ruff check pass in the current repository state.

A Summary of Public API

Signature	Module
<code>parse_netlist_text(text: str)</code> -> Circuit	<code>dnipro/dnipro.parser.netlist_parser</code>
<code>parse_netlist_file(path: str)</code> -> Circuit	<code>dnipro/dnipro.parser.netlist_parser</code>

```

build_equations(circuit,          dnipro/dnipro.mna.equation_builder
domain="time") ->
ResidualSystem
extract_linear_system(system)      dnipro.mna.linear_extraction
-> LinearSystem
extract_descriptor_system(system)  dnipro.mna.linear_extraction
-> DescriptorSystem
export_text(system) -> str         dnipro.export.text_exporter
export_json(system) -> str         dnipro.export.json_exporter
export_latex(system) -> str        dnipro.export.latex_exporter
export_linear_text(system) ->      dnipro.export.text_exporter
str
export_linear_json(system) ->      dnipro.export.json_exporter
str
export_linear_latex(system) ->     dnipro.export.latex_exporter
str
export_descriptor_text(system)     dnipro.export.text_exporter
-> str
export_descriptor_json(system)     dnipro.export.json_exporter
-> str
export_descriptor_latex(system)    dnipro.export.latex_exporter
-> str
apply_numeric_substitutions(system, dnipro.analysis.substitutions
subs) -> ResidualSystem
apply_frequency_substitution(system, dnipro.analysis.substitutions
*, freq_hz, omega) ->
ResidualSystem
collect_parameter_symbols(system)  dnipro.analysis.substitutions
-> set[Symbol]
parse_param_assignment(raw) ->     dnipro.analysis.substitutions
tuple[Symbol, Expr]
parse_param_assignments(rows)      dnipro.analysis.substitutions
-> dict[Symbol, Expr]
parse_value(raw: str) ->           dnipro.parser.value_parser
sympy.Expr
parse_source(raw: str) ->          dnipro.parser.source_parser
SourceFunction

```

B Reference Circuit Index

The following reference circuits are defined in the technical specification and are covered by tests in `tests/mna/test_reference_circuit_coverage.py` and `tests/integration`

ID	Circuit	Test purpose
TDD-CIR-001	Voltage divider	Basic KCL, voltage source constraint.
TDD-CIR-002	RC low-pass	Capacitor time derivative.
TDD-CIR-003	RC high-pass	Floating capacitor, non-ground nodes.

TDD-CIR-004	RL low-pass	Inductor branch current.
TDD-CIR-005	RLC low-pass	Second-order dynamics.
TDD-CIR-006	Current-driven RC	Current source sign convention.
TDD-CIR-007	VCVS amplifier	E source constraint.
TDD-CIR-008	VCCS transconductance	G source current direction.
TDD-CIR-009	CCCS current gain	F source, sensing branch.
TDD-CIR-010	CCVS transresistance	H source constraint.
TDD-CIR-011	SIN voltage source RC	Time-dependent V source.
TDD-CIR-012	SIN current source RC	Time-dependent I source.
TDD-CIR-013	Pulse-driven RC	PULSE source formula.
TDD-CIR-014	PWL-driven RC	PWL parser and formula.
TDD-CIR-015	Diode with resistor	Nonlinear residual formula.

C Version History

Version	Changes
0.1.0	Initial release. Supported elements: R, C, L, V, I, E, G, F, H, D. Source functions: DC, SIN, PULSE, PWL, SQUARE, SAW. Domains: time, Laplace. Exports: text, JSON, LaTeX. Linear reporting: residual form, $Ax = b$, and descriptor $Gx + C\dot{x} = b$. CLI: <code>dnipro analyze</code> , <code>--linear-system</code> , <code>--descriptor-system</code> . Documentation includes pub1 matrix reports and MOSFET small-signal tutorial verification against ngspice output.
